

Analytical Exam:

Given

$$w = x + y - z + 20$$

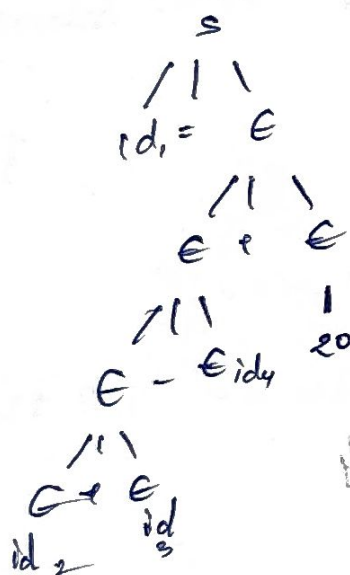
1) lexical analysis

$w \rightarrow$ Identifier, $=$ Assignment operator, $x \rightarrow$ Identifier, $+$ Multiplication factor,
 $y \rightarrow$ Identifier, $-$ minus operator, $z \rightarrow$ Identifier, $+$ plus operator, $20 \rightarrow$ Constant.
 $\langle id_1, w \rangle \Rightarrow \langle id_2, x \rangle \Rightarrow \langle id_3, y \rangle \Rightarrow \langle id_4, z \rangle \Rightarrow \langle 20 \rangle$

$$S \rightarrow id; E$$

$$E \rightarrow E + E \mid E - E \mid E * E \mid id / num.$$

Parse trees:



$$20 = (*, +) \quad (*, +, +)$$

$$\{ \dots, \dots, \dots, \dots, \dots \}$$

$$w$$

$$+$$

$$20$$

$$\dots$$

$$\dots$$

$$\dots$$

$$\dots$$

$$\dots$$

$$\dots$$

$$\dots$$

$$\dots$$

$$t_1 = id_1 * id_3$$

$$t_2 = t_1 - id_4$$

$$t_3 = t_2 + 20$$

$$w = t_3$$

optimise

$$t_1 = id_2 * id_3$$

$$t_2 = t_1 - id_4$$

$$w = t_2 + 20$$

codes

MOV id_1, R_1
 MOV id_3, R_2
 MUL R_1, R_2
 SUB R_1, id_4
 ADD $R_1, 20$
 MOV w, R_1

$$2) i. 1(0+1)^* 1 + 1$$

$$ii. (0+1)^* 0(0+1)^*$$

$$iii. 1^* 0 1^*$$

$$iv. (0+1)^* 1(0+1)^* 1(0+1)^*$$

$$v. \epsilon + 1 + (0+1)^*$$

$$8. i. (\epsilon + aa^*)^* = \epsilon + aa^*$$

$$(\epsilon + aa^*)^* \Rightarrow (a^*)^* = a^*$$

$$L = \{ \epsilon, a, aa, aaaa, \dots \}$$

$$ii. (a^* + b^*)^* abb$$

$$\underline{2.1} \quad (a^* + b^*)^* = (a + b)^* abb$$

Any combination of a & b.

$$L = \{ abb, aabb, babb, \dots \}$$

$$iii. (a^* b^*)^* aba$$

$$\underline{2.1} \quad (a + b)^* aba$$

Any combination of a & b.

$$L = \{ aba, aaba, baba, \dots \}$$

$$iv. (a + ab)^*$$

$$\underline{2.1} \quad L = \{ \epsilon, a, ab, aab, abab, \dots \}$$

$$v. (aba)^* + (a^* b^*)^*$$

$$\underline{2.1} \quad (a + b)^* = (a + b)^*$$

$$(aba)^* = \{ \epsilon, aba, abaaba, \dots \}$$

4. $\%d$
include <stdio.h>

$\%s$

*X. if (printf("keyword : if\n");)

else if (printf("keyword : else\n");)

while (printf("keyword : while\n");)

[0-9] + { printf("constant : %s\n", yytext); }

[a-zA-Z][a-zA-Z0-9]* { printf("identifier : %s\n", yytext); }

[+|-|*|/] { printf("operator : %s\n", yytext); }

[] { printf("invalid : %s\n", yytext); }

*X.

int main() {

yytext;

return 0;

}

printf("wrap\n");

return 1;

3.

with, [a-zA-Z][a-zA-Z0-9]*.

no. justification:

- starts with a letter

- Remaining characters can be letter/digits.

- Identifiers can't start with a digit.

- Keywords are matched before the identifier rule, so they are recognized as keywords instead of identifiers.

3. $(0^*1^*)^* = (0^*1)^*$

$$(0^*1^*)^* = ((0^*1)^*)^*$$

$$= 0^*1^*$$

both generate same language 'a' both are same

4. $(0^*1)^* (0^*1)^* = (0^*1)^*$

$$[r^* \cdot r^* = r^*]$$

Both generate same language. *Concentration gives language.*

5. $(0^*1)^* = 0^*$

$$[r^*r^* = r^*]$$

Double Kleene star is equal to single star

6. $(0^*1)^* \cup (0^*1)^* = (0^*1)^* ((0^*1)^* \cup (0^*1)^*)$

since every string contains combination of 0's & 1's, so both generate same language.

7. $\epsilon \cup 0(0^*)^* = 0^*$

$$\epsilon \cup r(r^*) = r^*$$

Since $rr^* = r^*r$ adding ϵ gives same r^*

$$[0(0^*)^* \cup \epsilon = 0^*]$$